Lecture 17 — Graphs II: Dijkstra, Bellman-Ford, Floyd-Warshall, 0-1 BFS

> Year Curriculum · Prerequisites: Week 13 (priority queues), Week 15 (DP), Week 16 (graph representations, BFS) · Estimated time: 3 hours

1. Overview

Last week BFS gave us shortest paths when every edge has the same cost. Real problems attach a **weight** to each edge — a travel time, a price, a penalty — and want the path of minimum total weight. Fewer edges is no longer the same as cheaper: in the graph with edges 1→2 (6), 1→3 (2), 3→2 (3), BFS would go straight 1→2 for cost 6, while 1→3→2 costs 5.

This lecture covers the four shortest-path algorithms every contestant must know:

Algorithm	Weights	Sources	Time
Dijkstra	≥ 0	single	O((n + m) log m)
Bellman-Ford	any; detects negative cycles	single	O(n·m)
Floyd-Warshall	any; detects negative cycles	all pairs	$O(n^3)$
0-1 BFS	0 or 1	single	O(n + m)

We also practise the most important skill around them: **modelling** a problem as a graph whose vertices are *states*.

2. Relaxation and the shape of a solution

All single-source algorithms maintain an upper bound `dist[v]` on the true distance δ(s, v): initially `dist[s] = 0` and `dist[v] = ∞` elsewhere. The only update is **relaxing** an edge u → v of weight w:

```
if dist[u] + w < dist[v]:  dist[v] = dist[u] + w;  par[v] = u
```

Relaxation never makes `dist[v]` smaller than the length of some real path, so it never undershoots δ. The algorithms differ only in *which* edges they relax and in *what order*, and the correctness proofs are arguments that the order is good enough to reach δ everywhere.

Two facts we will use:

- **Optimal substructure.** Any prefix of a shortest path is itself a shortest path. (Otherwise replace the prefix with a shorter one.)
- If there is no negative cycle, some shortest path is **simple**, so it has at most n − 1 edges.

3. Dijkstra's algorithm

3.1 Idea and correctness

Assume all weights are non-negative. Dijkstra keeps a set F of *finished* vertices whose `dist` is final. At every step it picks the unfinished vertex u with the smallest `dist[u]`, adds it to F, and relaxes all edges leaving u.

Why is `dist[u]` final at that moment? Take any s → u path P. It starts inside F (at s) and at some point leaves F for the first time, entering an unfinished vertex x. The prefix of P up to x has length at least `dist[x]` (x's predecessor on P is finished, and its edge to x was relaxed), and `dist[x] ≥ dist[u]` because u was chosen as the minimum. The remainder of P from x to u has non-negative weight. Hence |P| ≥ `dist[u]`. This argument uses **w ≥ 0** in its last step — which is exactly why Dijkstra fails with negative edges (§3.4).

3.2 Implementation with lazy deletion

`std::priority_queue` has no decrease-key operation. The standard workaround: whenever `dist[v]` improves, push a *new* pair `(dist[v], v)`; old pairs for v stay in the heap. When we pop a pair `(d, u)` with `d != dist[u]`, it is stale and we skip it.

```
priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
dist[s] = 0; pq.push({0, s});
while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d != dist[u]) continue;     // stale
    for (auto [v, w] : adj[u])
        if (dist[u] + w < dist[v]) { dist[v] = dist[u] + w; pq.push({dist[v], v}); }
}
```

Each successful relaxation pushes once, so the heap receives at most m + 1 entries, and each vertex is *processed* (edges scanned) exactly once thanks to the stale check. Total O((n + m) log m) = O(m log n) for connected graphs.

`priority_queue` is a **max**-heap by default; `greater<>` turns it into a min-heap. An equivalent trick is to push `{-dist, v}` into a max-heap.

Without the stale check the program is still correct but can rescan the adjacency list of a vertex once per copy in the heap; on adversarial tests (many improvements of a high-degree vertex) that is quadratic and times out.

3.3 Worked example — CSES 1671 *Shortest Routes I*

Edges: 1→2 (6), 1→3 (2), 3→2 (3), 1→3 (4).

pop	action	dist after
(0, 1)	relax 1→2: 6; 1→3: 2; 1→3 (4): no change	[0, 6, 2]
(2, 3)	relax 3→2: 2+3 = 5 < 6	[0, 5, 2]
(5, 2)	no outgoing edges	[0, 5, 2]
(6, 2)	stale (6 ≠ 5), skip	

Output `0 5 2`. Note the parallel edges 1→3 are handled automatically.

3.4 Negative edges break Dijkstra

Edges 1→2 (2), 2→3 (−5), 1→3 (1). Dijkstra pops 1, then 3 with `dist = 1` and declares it final; the true distance is 2 − 5 = −3. The lazy implementation happens to re-push 3 after popping 2 and would fix this small case, but in general a lazy "Dijkstra" with negative edges can run in exponential time. Do not use it: switch to Bellman-Ford, or — if the graph is a DAG — to DP in topological order (Week 21).

A related trap: "maximise the total" (longest path) is *not* Dijkstra with `max` instead of `min`. Longest simple path is NP-hard in general graphs; in a DAG it is easy (Week 21).

4. Bellman-Ford

4.1 Algorithm

Relax every edge, n − 1 times:

```
for (int round = 1; round <= n - 1; round++)
    for (auto [a, b, w] : edges)
        if (dist[a] != INF && dist[a] + w < dist[b]) dist[b] = dist[a] + w;
```

Invariant: after round k, `dist[v]` is at most the length of the shortest s → v path that uses at most k edges. (Induction: the shortest ≤ k-edge path is a shortest ≤ (k−1)-edge path plus one edge, which round k relaxes.) Since some shortest path has ≤ n − 1 edges when there are no negative cycles, n − 1 rounds suffice. Time O(n·m); stop early if a whole round changes nothing — on random data this is often much faster.

The `dist[a] != INF` guard matters with negative weights: without it, "∞ + (−5)" would make an unreachable vertex look reachable.

4.2 Detecting a negative cycle

If a negative cycle is reachable, distances along it decrease forever, so some relaxation succeeds in round n as well. Conversely, if round n relaxes nothing, the distances are a fixed point and there is no reachable negative cycle.

To **print** the cycle (CSES 1197 *Cycle Finding*): let x be any vertex relaxed in round n. Its parent chain eventually enters the cycle, but x itself might hang off it. Moving x ← par[x] n times guarantees we are on the cycle (the chain has at most n distinct vertices before repeating). Then follow `par` from x until x reappears and reverse the list.

To find a negative cycle *anywhere* (not only reachable from a particular s), initialise `dist[v] = 0` for every v. This simulates a virtual source with 0-weight edges to all vertices.

4.3 When a negative cycle matters

In CSES 1673 *High Score* we maximise a score from room 1 to room n; negating weights turns it into a shortest path. If a negative cycle exists but is unreachable from 1, or cannot reach n, the answer is still finite. The correct test: mark every vertex relaxed in round n (and, to be safe, everything reachable from those), then output −1 only if some marked vertex lies on a 1 → n route — i.e. is reachable from 1 *and* can reach n (BFS on the graph and on the reversed graph).

5. Floyd-Warshall

5.1 The DP

Number the vertices 1..n. Let d_k[i][j] be the length of the shortest i → j path whose *intermediate* vertices all lie in {1, …, k}. Either the path avoids k, or it passes through k exactly once (with no negative cycles, a shortest path is simple):

```
d_k[i][j] = min( d_{k-1}[i][j],  d_{k-1}[i][k] + d_{k-1}[k][j] )
```

d_0 is the edge weights (0 on the diagonal, ∞ for non-edges), and d_n is the answer. This is precisely a Week 15-style DP with an extra dimension k that we roll away. Rolling is safe because in round k the entries `d[i][k]` and `d[k][j]` do not change (using k as an intermediate vertex cannot improve a path that ends at k).

```
for (int k = 1; k <= n; k++)
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
```

The loop over **k must be outermost**. With k innermost, d[i][j] is finalised before the paths through later intermediates have been computed, and the result is wrong.

5.2 Details

- $O(n^3)$ time, $O(n^2)$ memory: n = 500 means $1.25 \cdot 10^8$ very simple operations, fine. n = 2000 is not.
- Initialise with `d[a][b] = min(d[a][b], w)` — inputs often contain several edges between the same pair.
- With negative edges, guard against `INF + negative`, e.g. skip when `d[i][k] == INF`.
- A negative cycle exists iff some `d[i][i] < 0` at the end.
- Variants of the same triple loop: transitive closure (`reach[i][j] |= reach[i][k] && reach[k][j]`), minimax / bottleneck paths (`min(max(...))`).

CSES 1672 *Shortest Routes II* ($n \le 500$, up to 10^5 queries) is the canonical use: precompute all pairs once, answer each query in O(1).

6. 0-1 BFS

When every weight is 0 or 1, Dijkstra's heap can be replaced by a **deque**. Pop from the front; when relaxing with a 0-edge, push to the front (same distance); with a 1-edge, push to the back (distance + 1). The deque always contains vertices with distance d at the front and d + 1 at the back, mirroring the BFS layer invariant, so vertices leave the deque in non-decreasing order of distance just as in Dijkstra. Runtime O(n + m).

A vertex may be pushed more than once (once for a 1-edge, later improved through a 0-edge). That is harmless: the later, better copy is popped first; the stale copy causes at most a rescan that relaxes nothing. You may add the same `d != dist[u]` style check if you store distances in the deque.

Modelling example — CodeChef REVERSE. Given a directed graph, find the minimum number of edges to reverse so that vertex n is reachable from vertex 1. For every edge u → v add an edge u → v of weight 0 (use it as it is) and v → u of weight 1 (use it backwards, paying one reversal). The 0-1 BFS distance from 1 to n is the answer. The same pattern solves grid problems where moving is free but breaking a wall, or changing direction, costs one (AtCoder ABC 213 E *Stronger Takahashi*, CF 1063B *Labyrinth* in a variant).

7. Modelling: shortest paths on state graphs

Many "shortest path with a twist" problems become ordinary Dijkstra once the vertex is a *state* that carries the twist:

- **CSES 1195 *Flight Discount*:** you may halve one flight's price once. State (city, used ∈ {0, 1}); from (u, 0) you can go to (v, 0) for w or to (v, 1) for w/2; from (u, 1) only to (v, 1) for w. Run Dijkstra on 2n vertices.
- **Fuel or remaining tickets:** state (city, amount left), as long as the amount has a small range.
- **CSES 1202 *Investigation*:** along with `dist`, maintain the number of shortest paths, the minimum and the maximum number of edges among them, updating them when `dist[u] + w == dist[v]` (tie) or `<` (improvement). Correct because Dijkstra processes vertices in order of distance.

Ask: "what do I need to know about my current situation to decide the next move?" — that tuple is the vertex.

8. Reference implementations

All compile with `g++ -std=c++17 -O2 -Wall` and pass the problem samples.

8.1 Dijkstra — CSES 1671 *Shortest Routes I*

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = LLONG_MAX / 4;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<pair<int, int>>> adj(n + 1);   // (neighbour, weight)
    for (int i = 0; i < m; i++) {
        int a, b, c;
        cin >> a >> b >> c;
        adj[a].push_back({b, c});
    }
    vector<ll> dist(n + 1, INF);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq; // min-heap
    dist[1] = 0;
    pq.push({0, 1});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d != dist[u]) continue;              // stale entry: skip
        for (auto [v, w] : adj[u])
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
    }
    for (int v = 1; v <= n; v++) cout << dist[v] << " ";
```

```
    cout << "\n";
}
```

8.2 Bellman-Ford with negative-cycle extraction — CSES 1197 *Cycle Finding*

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<array<ll, 3>> edges(m);
    for (auto &[a, b, c] : edges) cin >> a >> b >> c;
    // dist = 0 everywhere acts like a virtual source joined to every vertex,
    // so a negative cycle anywhere in the graph is detected.
    vector<ll> dist(n + 1, 0);
    vector<int> par(n + 1, 0);
    int last = -1;                                  // vertex relaxed in round n
    for (int round = 1; round <= n; round++) {
        last = -1;
        for (auto [a, b, c] : edges)
            if (dist[a] + c < dist[b]) {
                dist[b] = dist[a] + c;
                par[b] = a;
                last = b;
            }
    }
    if (last == -1) { cout << "NO\n"; return 0; }
    // 'last' may hang off the cycle; walking back n steps lands inside it.
    int x = last;
    for (int i = 0; i < n; i++) x = par[x];
    vector<int> cyc;
    for (int v = x;; v = par[v]) {
        cyc.push_back(v);
        if (v == x && cyc.size() > 1) break;
    }
    reverse(cyc.begin(), cyc.end());
    cout << "YES\n";
    for (int v : cyc) cout << v << " ";
    cout << "\n";
}
```

On the CSES sample (edges 1→2 1, 2→4 1, 3→1 1, 4→1 −3, 4→3 −2) it prints `YES` and `4 1 2 4`, a cycle of weight −1.

8.3 Floyd-Warshall — CSES 1672 *Shortest Routes II*

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = LLONG_MAX / 4;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    vector<vector<ll>> d(n + 1, vector<ll>(n + 1, INF));
    for (int i = 1; i <= n; i++) d[i][i] = 0;
    for (int i = 0; i < m; i++) {
        ll a, b, c;
        cin >> a >> b >> c;
        d[a][b] = min(d[a][b], c);              // keep the cheapest parallel edge
        d[b][a] = min(d[b][a], c);
    }
    for (int k = 1; k <= n; k++)                // k MUST be the outer loop
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= n; j++)
                if (d[i][k] + d[k][j] < d[i][j])
                    d[i][j] = d[i][k] + d[k][j];
    while (q--) {
        int a, b;
        cin >> a >> b;
        cout << (d[a][b] >= INF ? -1 : d[a][b]) << "\n";
    }
}
```

`INF = LLONG_MAX / 4` makes `INF + INF` safe from overflow, and weights here are non-negative so no extra guard is needed.

8.4 0-1 BFS — CodeChef REVERSE

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<pair<int, int>>> adj(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back({v, 0});
        adj[v].push_back({u, 1});
    }
    const int INF = INT_MAX;
```

```
    vector<int> dist(n + 1, INF);
    deque<int> dq;
    dist[1] = 0;
    dq.push_back(1);
    while (!dq.empty()) {
        int u = dq.front(); dq.pop_front();
        for (auto [v, w] : adj[u])
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                if (w == 0) dq.push_front(v);    // same layer: front
                else dq.push_back(v);            // next layer: back
            }
    }
    cout << (dist[n] == INF ? -1 : dist[n]) << "\n";
}
```

Sample (7 vertices, edges 1→2, 3→2, 3→4, 7→4, 6→2, 5→6, 7→5) → `2`.

9. Common mistakes

- **Overflow.** $2 \cdot 10^5$ edges of weight 10^9 sum to $2 \cdot 10^{14}$: distances must be `long long`. Choose `INF` so that `INF + w` (and, for Floyd-Warshall, `INF + INF`) does not overflow.
- **Max-heap instead of min-heap** — the program still terminates and gives wrong answers on most tests.
- **Missing stale check** — correct but TLE on crafted tests.
- **Dijkstra with negative edges** — wrong or exponential.
- **Floyd-Warshall loop order** — k must be outermost.
- **Parallel edges** — taking the last edge read rather than the minimum when filling a matrix.
- **Undirected input** — add both directions.
- **Negative cycle "anywhere"** vs **"on the route"** — read the question; High Score is the classic trap.
- **Bellman-Ford with `INF` guard missing** — unreachable vertices get finite distances with negative edges.
- **Not resetting `dist`**, heaps or deques between test cases.

10. Practice problems

- CSES 1671 — Shortest Routes I — https://cses.fi/problemset/task/1671
- CSES 1672 — Shortest Routes II — https://cses.fi/problemset/task/1672
- CSES 1195 — Flight Discount — https://cses.fi/problemset/task/1195
- CSES 1202 — Investigation — https://cses.fi/problemset/task/1202
- CSES 1673 — High Score — https://cses.fi/problemset/task/1673
- CSES 1197 — Cycle Finding — https://cses.fi/problemset/task/1197
- Codeforces 20C — Dijkstra? — https://codeforces.com/problemset/problem/20/C
- Codeforces 1063B — Labyrinth — https://codeforces.com/problemset/problem/1063/B

- AtCoder ABC 213 E — Stronger Takahashi — https://atcoder.jp/contests/abc213/tasks/abc213_e
- CodeChef REVERSE — Chef and Reversing — https://www.codechef.com/problems/REVERSE
- Kattis — Single source shortest path, non-negative weights — https://open.kattis.com/problems/shortestpath1

11. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 13 (Shortest paths).
- CP-Algorithms: "Dijkstra on sparse graphs" (https://cp-algorithms.com/graph/dijkstra_sparse.html), "Bellman-Ford" (https://cp-algorithms.com/graph/bellman_ford.html), "Floyd-Warshall" (https://cp-algorithms.com/graph/all-pair-shortest-path-floyd-warshall.html), "0-1 BFS" (https://cp-algorithms.com/graph/01_bfs.html).
- Looking ahead: Week 21 handles shortest and longest paths in DAGs in linear time; the Regional track (Graph modeling lecture) goes deeper into state-space Dijkstra.